

Activités : remplacer dependance par collaborateur_id

1. Film mental et objectif

Visualisez une activité rattachée à un utilisateur autorisé : création, lecture, modification puis effacement du responsable. Avancez par étapes observables, sans forcer une conclusion avant les résultats.

2. Architecture et invariants

Le contrat JSON expose exactement collaborateur_id (un seul). Il référence accounts.Utilisateur. Le modèle utilise collaborateur et Django conserve son UUID dans collaborateur_id. Le champ est facultatif, nullable, avec SET_NULL. Les anciennes dépendances restent en base : aucune conversion des UUID activité vers les UUID utilisateur.

3. Implémentation pratique

Dans le modèle Activite, ajouter la ForeignKey nullable. Dans le serializer de lecture, déclarer PrimaryKeyRelatedField(source="collaborateur", read_only=True). Dans le serializer de création et de PATCH, déclarer le même source avec queryset utilisateur actif et non supprimé. Remplacer dependance dans Meta.fields. Supprimer les validations de dépendance des entrées API. Valider la présence du collaborateur dans les affectations actives du projet, ou parmi son chef de projet et conducteur de travaux. equipe_ids conserve sa fonction indépendante.

4. Migration et données

La migration 0026 ajoute uniquement collaborateur ; les activités existantes ont null. Aucun responsable ne peut être déduit de leurs anciennes dépendances. Vérifier la migration en environnement de test puis appliquer les migrations tenant avec python manage.py migrate_schemas. La suppression logique des activités liées historiquement reste protégée par les règles existantes.

5. Erreurs et pré-mortem

UUID invalide, utilisateur inconnu, inactif ou supprimé : erreur de validation. Utilisateur non affecté au projet : 400. Champ dependance transmis : 400 Champ non accepté. Contrôler les droits du projet en plus de la validité du responsable. Un PATCH omettant collaborateur_id préserve sa valeur ; null la supprime. Ne pas changer les UUID historiques. Le contrat API change : les clients doivent envoyer le nouveau nom.

6. Tests et validation

Tests de validation sans base : création du champ, portée du projet, responsable autorisé, PATCH conservant la relation, null et rejet de dépendance. Tests d'intégration : POST, GET, PATCH, statut 400 hors projet et schéma Swagger. Exécuter pytest apps/projets/tests/test_lots_activites_validation.py -q puis les tests formulaire, mutations et Swagger avec PostgreSQL disponible. Vérifier makemigrations --check --dry-run.

7. Consolidation et défi Homo Docens

Les quatre piliers : attention au contrat, engagement par un exemple JSON, retour d'erreur par les tests, consolidation en reformulant demain. Distinguez hypothèse et preuve dans votre diagnostic. Expliquez ? un pair pourquoi renommer une relation vers Activite ne peut pas la transformer en relation vers Utilisateur, puis illustrez source="colaborateur". Cette explication constitue le d'fi de transmission.

```
{"libelle": "Terrassement",
  "colaborateur_id": "<UUID utilisateur>"}

PATCH : {"colaborateur_id": null}
```

R?f?rence : apps/projets/models/activite.py

```
colaborateur = models.ForeignKey(
    "accounts.Utilisateur",
    null=True,
    blank=True,
    on_delete=models.SET_NULL,
    related_name="activites_responsables",
)
```

R?f?rence : apps/projets/serializers/activite.py

```
"""Serializers pour la gestion des activités de lot (MLD §6.3)."""
from decimal import Decimal
from rest_framework import serializers
from apps.accounts.models import Utilisateur
from apps.core.enums import StatutUtilisateur, UniteMesure
from apps.projets.models import Activite, AffectationProjet
from apps.projets.services.statistiques import pourcentage_realise
class ActiviteSerializer(serializers.ModelSerializer):
    """Lecture complète d'une activité de chantier."""
    colaborateur_id = serializers.PrimaryKeyRelatedField(source="colaborateur", read_only=True)
    unite_libelle = serializers.CharField(source="get_unite_display", read_only=True)
    statut = serializers.CharField(read_only=True)
    avancement = serializers.SerializerMethodField()
    equipe_ids = serializers.PrimaryKeyRelatedField(source="equipe", many=True, read_only=True)
    def get_avancement(self, obj) -> float:
        return float(round(pourcentage_realise(obj), 2))
class Meta:
    model = Activite
    fields = [
        "id",
        "lot_id",
        "libelle",
        "motif",
        "statut",
        "colaborateur_id",
        "equipe_ids",
        "budget_initial_montant",
        "unite",
        "unite_libelle",
        "quantite_prevue",
        "quantite_realisee",
        "avancement",
        "poids",
        "date_debut_prevue",
        "date_fin_prevue",
        "date_debut_baseline",
        "date_fin_baseline",
        "ordre",
        "est_actif",
        "cree_le",
    ]
    read_only_fields = [
        "id",
        "lot_id",
        "quantite_realisee",
        "avancement",
        "date_debut_baseline",
        "date_fin_baseline",
        "cree_le",
    ]
class ActiviteCreationSerializer(serializers.ModelSerializer):
```

```

"""Création d'une activité rattachée à un lot."""
motif = serializers.CharField(
    required=False,
    allow_blank=True,
    trim_whitespace=False,
    help_text="Motif informatif facultatif ; texte libre, chaîne vide pour l'effacer.",
)
statut = serializers.CharField(
    required=False,
    trim_whitespace=False,
    help_text="Statut d'évolution libre envoyé par le frontend ; chaîne non vide.",
)
quantite_prevue = serializers.DecimalField(
    max_digits=14,
    decimal_places=3,
    min_value=Decimal("0.001"),
    required=False,
    default=Decimal("1.000"),
)
budget_initial_montant = serializers.IntegerField(
    required=False,
    allow_null=True,
    min_value=0,
    max_value=9223372036854775807,
    help_text="Budget prévisionnel facultatif en centimes FCFA, utilisé pour la
pondération.",
)
colaborateur_id = serializers.PrimaryKeyRelatedField(
    source="colaborateur",
    queryset=Utilisateur.objects.filter(is_active=True, statut=StatutUtilisateur.ACTIF,
supprime_le__isnull=True),
    required=False,
    allow_null=True,
    help_text="UUID d'un utilisateur actif affecté au projet, ou null.",
)
equipe_ids = serializers.PrimaryKeyRelatedField(
    source="equipe",
    many=True,
    required=False,
    queryset=Utilisateur.objects.filter(is_active=True, statut=StatutUtilisateur.ACTIF),
    help_text="UUID des collaborateurs actifs affectés au projet ; facultatif.",
)
class Meta:
    model = Activite
    fields = [
        "libelle",
        "motif",
        "statut",
        "colaborateur_id",
        "equipe_ids",
        "budget_initial_montant",
        "unite",
        "quantite_prevue",
        "poids",
        "date_debut_prevue",
        "date_fin_prevue",
        "ordre",
    ]
def to_internal_value(self, data):
    inconnus = set(data.keys()) - set(self.fields)
    if inconnus:
        raise serializers.ValidationError(dict.fromkeys(inconnus, "Champ non accepté.))
    return super().to_internal_value(data)
def validate(self, attrs):
    lot = self.context.get("lot")
    if not lot:
        raise serializers.ValidationError("Le lot de rattachement est requis.")
    debut = attrs.get("date_debut_prevue")
    fin = attrs.get("date_fin_prevue")
    if debut and fin and fin < debut:
        raise serializers.ValidationError(
            {"date_fin_prevue": "La fin prévue ne peut pas précéder le début."}
        )
    equipe = attrs.get("equipe", [])
    if len(equipe) > 50:
        raise serializers.ValidationError({"equipe_ids": "Maximum 50 collaborateurs."})
    autorises = set(

```

```

        AffectationProjet.objects.filter(
            projet_id=lot.projet_id,
            est_actif=True,
        ).values_list("utilisateur_id", flat=True)
    )
    autorises.update((lot.projet.chef_projet_id, lot.projet.conducteur_travaux_id))
    collaborateur = attrs.get("collaborateur")
    if collaborateur is not None and collaborateur.pk not in autorises:
        raise serializers.ValidationError(
            {"collaborateur_id": "Collaborateur non affecté au projet."}
        )
    if any(user.pk not in autorises for user in equipe):
        raise serializers.ValidationError(
            {"equipe_ids": "Collaborateur non affecté au projet."}
        )
    # Vérification des bornes du Lot parent
    if lot.date_debut_prevue and debut and debut < lot.date_debut_prevue:
        raise serializers.ValidationError(
            {
                "date_debut_prevue": (
                    f"L'activité ne peut pas débuter avant la date de début du lot "
                    f"({lot.date_debut_prevue.strftime('%d/%m/%Y')})."
                )
            }
        )
    if lot.date_fin_prevue and fin and fin > lot.date_fin_prevue:
        raise serializers.ValidationError(
            {
                "date_fin_prevue": (
                    f"L'activité ne peut pas se terminer après la date de fin du lot "
                    f"({lot.date_fin_prevue.strftime('%d/%m/%Y')})."
                )
            }
        )
    unite = attrs.get("unite", UniteMesure.UNITE)
    quantite = attrs.get("quantite_prevue")
    if unite == UniteMesure.FORFAIT and quantite != Decimal("1.000"):
        attrs["quantite_prevue"] = Decimal("1.000")
    return attrs
def create(self, validated_data):
    lot = self.context["lot"]
    request = self.context.get("request")
    utilisateur = getattr(request, "user", None)
    equipe = validated_data.pop("equipe", [])
    activite = Activite.objects.create(
        lot=lot,
        cree_par=utilisateur,
        **validated_data,
    )
    activite.equipe.set(equipe)
    return activite
class ActiviteModificationSerializer(ActiviteCreationSerializer):
    """Réutilise la validation métier sur l'état final d'une modification partielle."""
    def validate(self, attrs):
        for champ in ("date_debut_prevue", "date_fin_prevue"):
            ancienne = getattr(self.instance, champ)
            if champ in attrs and ancienne is not None and attrs[champ] != ancienne:
                raise serializers.ValidationError(
                    {champ: "RG-11 : reprogrammez l'activité avec motif et justification."}
                )
        valeurs = {
            champ.source: getattr(self.instance, champ.source)
            for champ in self.fields.values()
            if champ.source != "equipe"
        }
        valeurs.update(attrs)
        valeurs = super().validate(valeurs)
        if valeurs["quantite_prevue"] < self.instance.quantite_realisee:
            raise serializers.ValidationError(
                {"quantite_prevue": "Quantité inférieure au réalisé."}
            )
        fin = valeurs.get("date_fin_prevue")
        if fin and self.instance.successeurs.filter(date_debut_prevue__lt=fin).exists():
            raise serializers.ValidationError(
                {"date_fin_prevue": "La fin dépasse le début d'une activité suivante."}
            )
    # FORFAIT normalise la quantité à 1 ; les autres valeurs absentes restent intactes.

```

```

    if valeurs["quantite_prevue"] != self.instance.quantite_prevue:
        attrs["quantite_prevue"] = valeurs["quantite_prevue"]
    return attrs

```

Référence :

apps/projets/migrations/0026_activite_colaborateur.py

```

from django.conf import settings
from django.db import migrations, models
import django.db.models.deletion
class Migration(migrations.Migration):
    dependencies = [
        ("projets", "0025_motif_lots_activites"),
        migrations.swappable_dependency(settings.AUTH_USER_MODEL),
    ]
    operations = [
        migrations.AddField(
            model_name="activite",
            name="colaborateur",
            field=models.ForeignKey(
                to=settings.AUTH_USER_MODEL, null=True, blank=True,
                on_delete=django.db.models.deletion.SET_NULL,
                related_name="activites_responsables",
            ),
        ),
    ]

```